فصل ۲: گردش کار و پیش پردازش

مقدمه

داده چیست و ما با آن چه کار می‌کنیم؟ یک پاسخ ساده این است که ما سعی می‌کنیم داده‌های خود را به عنوان نقاطی روی کاغذ قرار دهیم، آن‌ها را رسم کنیم، فکر کنیم، و به دنبال توضیحات ساده‌ای باشیم که داده‌ها را به خوبی تخمین بزنند. این فصل به پیش پردازش داده‌ها در یادگیری می‌پردازد. برخی از سوالاتی که می‌توان در مورد مجموعه داده پرسید به شرح زیر است:

آیا مقادیر از دست رفته (missing value) در مجموعه داده شما وجود دارد؟

آیا در مجموعه شما داده‌های پرت (outlier) وجود دارد؟

متغیرهای داده چگونه هستند؟ آیا مقادیر پیوسته هستند یا گسسته هستند؟

توزیع‌های متغیر پیوسته چگونه هستند؟ آیا متغیرها در با توزیع‌های نرمال (منحنی‌های زنگ‌دار) توصیف می‌شود؟

آیا می‌توان متغیرهای پیوسته را برای سادگی به متغیرهای گسسته تبدیل کرد؟

واحدهای متغیرها کدامند؟ آیا متغیرها را مخلوط می‌کنید؟

این سوالات می‌توانند پاسخ‌های ساده یا پیچیده‌ای داشته باشند. در ادامه خطوط لوله را بررسی می‌کنیم: یک ابزار عالی برای اطمینان از اینکه عملیات مشابه در هر دو مجموعه آموزش (training) و آزمایش (test) بدون خطا و با کار نسبتاً کم انجام می‌شود و در ادامه به نمونه‌های رگرسیون خواهیم پرداخت.

ایجاد نمونه داده برای تجزیه و تحلیل

در صورت امکان، از داده‌های خود برای این کتاب استفاده کنید، در غیر این صورت، ما خواهیم آموخت که چگونه می‌توانیم از scikit-learn برای ایجاد داده‌ها استفاده کنیم. داده‌های ساخته شده در scikit-learn بسیار جالب است. داده‌های مصنوعی ساخته شده از قرارداد نامگذاری *_make استفاده می‌کنند.

```
from sklearn import datasets
datasets.make?*_
datasets.make_biclusters
datasets.make_blobs
datasets.make_checkerboard
datasets.make_circles
datasets.make_classification
```

برای راحتی می‌توان به ماژول نام مستعار داد و از نوشتن نام‌های طولانی جلوگیری نمود مانند:

```
import sklearn.datasets as d
import numpy as np
d.make_*?
d.make_biclusters
d.make_blobs
d.make_checkerboard
d.make_circles
d.make_classification
```

چگونه این کار انجام می‌شود

در این بخش با ایجاد چندین مجموعه داده آشنا خواهید شد. علاوه بر مجموعه داده‌های نمونه، داده‌هایی با مشخصات لازم برای انواع الگوریتم‌ها توضیح داده می‌شود.

ساخت مجموعه داده برای رگراسیون

دستور زیر

```
reg_data = d.make_regression()
```

به طور پیش‌فرض، این یک tuple با یک ماتریس ۱۰۰×۱۰۰ تولید می‌کند - ۱۰۰ نمونه در ۱۰۰ ویژگی. با این حال، به طور پیش‌فرض، فقط ۱۰ ویژگی مسئول تولید داده‌های

هدف هستند و عضو دوم تاپل متغیر هدف است. به عنوان مثال، برای ایجاد یک ماتریس ۱۰ × ۱۰۰۰ با پنج ویژگی اصلی، یک عامل بایاس ۱ و ۲ هدف مانند زیر است:

```
complex_reg_data = d.make_regression(1000, 10, 5, 2, 1.0)
complex_reg_data[0].shape
```

دستور اصلی به صورت زیر می‌باشد

```
#sklearn.datasets.make_regression(n_samples=100, n_features=100, *, n_infor
mative=10,
 n_targets=1, bias=0.0)
```

ایجاد یک مجموعه داده دسته‌بندی نامتعادل

برای دسته‌بندی بهتر است از داده‌های نامرتب استفاده گردد

```
classification_set = d.make_classification(weights=[0.1])
np.bincount(classification_set[1])
```

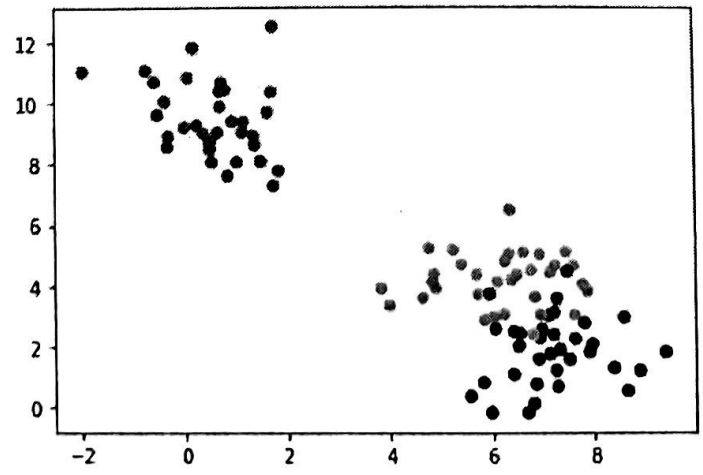
مقدار weights نسبت نمونه‌های اختصاص داده شده به هر کلاس را تعیین می‌کند. اگر مقداری تعیین نگردد، کلاس‌ها متعادل هستند.

به طور پیش فرض تعداد نمونه‌ها ۱۰۰، تعداد ویژگی‌ها ۲۰ و تعداد کلاس‌ها ۲ می‌باشد.

ایجاد یک مجموعه داده برای خوشه‌بندی

```
blobs_data, blobs_target = d.make_blobs()
import matplotlib.pyplot as plt
plt.scatter(blobs_data[:,0],blobs_data[:,1],c = blobs_target)
```

خروجی در زیر آورده شده است.



شکل (۲-۱): یک مجموعه داده برای خوشه بندی

نکته:
Matplotlib برای رسم نمودار و نمایش تصاویر استفاده می‌گردد.

تبدیل داده‌ها به استاندارد توزیع نرمال

در پیش پردازش تبدیل ستون‌ها به حالت استاندارد توصیه شده است و نرمال استاندارد احتمالاً مهم‌ترین توزیع در آمار است.

اگر تا به حال با آمار آشنا باشید، حتماً z-scores را دیده‌اید. فرمول محاسبه آن به صورت زیر می‌باشد

$$z = \frac{x - \mu}{\sigma}$$

x مقدار، μ میانگین داده‌ها و σ انحراف معیار می‌باشد.

عمل مقیاس‌گذاری داده‌ها بسیار مفید است. الگوریتم‌های یادگیری ماشین زیادی وجود دارد، که در صورت وجود ویژگی‌ها در مقیاس‌های مختلف، متفاوت (و نادرست) عمل می‌کنند. به عنوان مثال، اگر داده‌ها مقیاس‌بندی نشده باشند، SVM عملکرد ضعیفی دارند، زیرا آن‌ها از فاصله در بهینه‌سازی خود استفاده می‌کنند.

ماژول preprocessing شامل چندین تابع مفید برای ویژگی‌های مقیاس‌گذاری است. برای نمایش عملکرد این ماژول از داده‌های boston استفاده می‌کنیم

```
from sklearn import preprocessing
import numpy as np
from sklearn.datasets import load_boston
boston = load_boston()
X,y = boston.data, boston.target
```

حال میانگین ۳ ستون اول را بدست می‌آوریم

```
X[:, :3].mean(axis=0)
Output: array([ 3.61352356, 11.36363636, 11.13677866])
```

انحراف معیار همان ۳ ستون را نیز به دست می‌آوریم

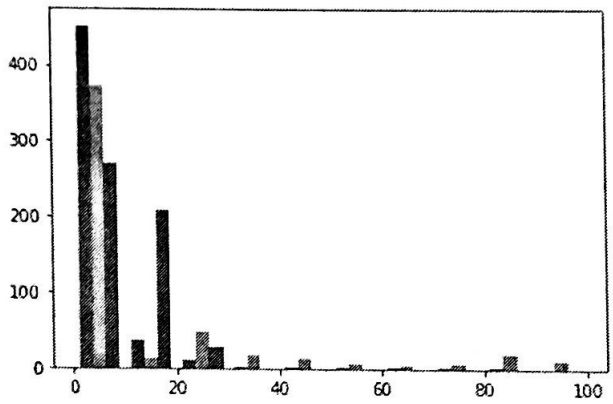
```
X[:, :3].std(axis=0)
Output: array([ 8.59304135, 23.29939569,  6.85357058])
```

با توجه به خروجی‌ها (output) اولین ویژگی کمترین میانگین را دارد و ویژگی دوم دارای بیشترین میانگین و انحراف معیار است. یعنی بیشترین پراکندگی مقادیر را به خود اختصاص داده است. حال آن‌ها را نرمال می‌کنیم تا ببینیم چه تغییری در میانگین و انحراف معیار ایجاد می‌شود.

```
X_2 = preprocessing.scale(X[:, :3])
X_2.mean(axis=0)
Output: array([-8.78743718e-17, -6.34319123e-16, -2.68291099e-15])
X_2.std(axis=0)
Output: array([1., 1., 1.])
```

اگر نمودار داده‌های اولیه را رسم کنیم (با ۳ ویژگی) شکل زیر حاصل می‌شود

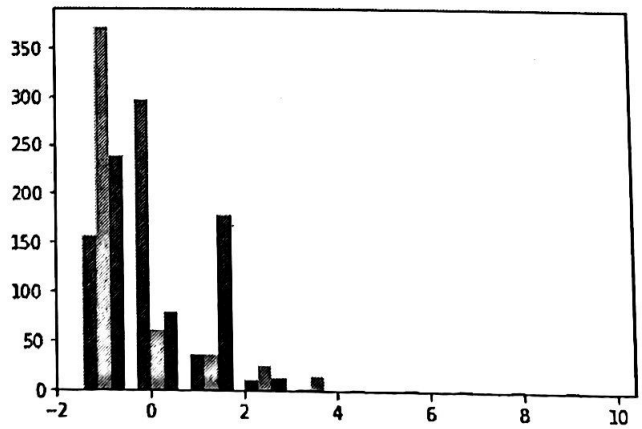
```
plt.hist(X[:, :3])
plt.show()
```



شکل (۲-۲): توزیع نرمال

و اگر نمودار داده‌های استاندارد شده را رسم کنیم (با ۳ ویژگی) شکل زیر حاصل می‌شود

```
plt.hist(X_2)
plt.show()
```



شکل (۲-۳): توزیع نرمال

علاوه بر روش قبلی، یک کلاس دیگر نیز وجود دارد که فراخوانی آن آسان است و کاربر آن در pipelineها بسیار مشهود است.

```
my_scaler = preprocessing.StandardScaler()
my_scaler.fit(X[:, :3])
my_scaler.transform(X[:, :3]).mean(axis=0)
array([-8.78743718e-17, -6.34319123e-16, -2.68291099e-15])
```

همان‌طور که مشاهده می‌کنید خروجی مانند روش قبلی است

پیش پردازش همچنین شامل کلاس MinMaxScaler است، که داده‌ها را در یک محدوده خاص مقیاس‌بندی می‌کند

```
my_minmax_scaler = preprocessing.MinMaxScaler()
my_minmax_scaler.fit(X[:, :3])
my_minmax_scaler.transform(X[:, :3]).max(axis=0)
output: array([1., 1., 1.])
my_minmax_scaler.transform(X[:, :3]).min(axis=0)
output:array([0., 0., 0.])
```

تغییر حداقل و حداکثر مقادیر MinMaxScaler از پیش‌فرض‌های ۰ و ۱ بسیار ساده است

```
my_odd_scaler = preprocessing.MinMaxScaler(feature_range=(-3.14, 3.14))
```

علاوه بر این، گزینه دیگر **نرمال‌سازی** است که هر نمونه را به یک طول مقیاس‌بندی می‌کند. این با انواع دیگر مقیاس‌گذاری که قبلاً مقیاس‌بندی شده بودند، متفاوت است. نرمال‌سازی در دستور زیر نشان داده شده است

```
normalized_X = preprocessing.normalize(X[:, :3])
```

فرمول ریاضی آن به صورت

$$x_{new} = \frac{x_i - Min(x)}{Max(x) - Min(x)}$$

ایجاد ویژگی‌های باینری از طریق آستانه

در آخرین دستورالعمل، ما به دنبال تبدیل داده‌های خود به توزیع استاندارد بودیم. اکنون، ما در مورد یک تغییر شکل دیگر صحبت خواهیم کرد، که کاملا متفاوت است. به جای استاندارد کردن داده‌ها، ما داده‌ها را عمداً دور می‌ریزیم. اگر دلیل خوبی برای این کار وجود داشته باشد، این عمل می‌تواند یک حرکت بسیار هوشمندانه باشد. غالباً در داده‌های به ظاهر پیوسته، ناپیوستگی‌هایی وجود دارد که می‌تواند از طریق ویژگی‌های **باینری** مشخص گردند.

نکته:

به کمک آستانه، می‌توانیم یک مسئله رگرسیون را به یک مسئله دسته‌بندی تبدیل کنیم. این اتفاق در برخی زمینه‌های علوم داده رخ می‌دهد.

ایجاد ویژگی‌ها و نتایج باینری یک روش بسیار مفید است، اما باید با احتیاط استفاده شود. بیایید از مجموعه داده بوستون برای تبدیل مقادیر به نتایج باینری استفاده کنیم. از دو تابع preprocessing.binariz و preprocessing.Binarizer برای این کار استفاده می‌کنیم.

متغیر هدف مجموعه داده بوستون میانه ارزش خانه‌ها در هزاران واحد است. این مجموعه داده برای آزمایش رگرسیون و سایر پیش‌بینی‌های مداوم خوب است، اما شرایطی را در نظر بگیرید که می‌خواهیم به راحتی پیش‌بینی کنیم که آیا ارزش یک خانه بیش از ارزش کلی است.

ما یک مقدار آستانه از میانگین ایجاد می‌کنیم. اگر مقدار بیشتر از میانگین باشد، ۱ و اگر کمتر باشد، صفر در نظر می‌گیریم.

```
import numpy as np
from sklearn.datasets import load_boston
from sklearn import preprocessing
boston = load_boston()
X, y = boston.data, boston.target.reshape(-1, 1)
new_target =preprocessing.binarize(y,threshold=boston.target.mean())
new_target[:5]
output: array([[1.],     [0.],     [1.],     [1.],     [1.]])
```

اما بیایید بررسی کنیم که آیا درست کار کرده است

```
(y[:5] > y.mean()).astype(int)
Output: array([[1],     [0],     [1],     [1],     [1]])
```

حال از کلاس Binarizer استفاده می‌کنیم

```
binar = preprocessing.Binarizer(y.mean())
new_target = binar.fit_transform(y)
new_target[:5]
output:
array([[1.],     [0.],     [1.],     [1.],     [1.]])
```

ماتریس‌های پراکنده (sparse)

در ماتریس‌های پراکنده تعداد صفرهای زیادی وجود دارد بنابراین صفرها را ذخیره نمی‌کنند. این کار با تلاش برای صرفه جویی در فضای حافظه انجام می‌شود. این مسئله مشکلی را برای binarizer ایجاد می‌کند، بنابراین، یک شرط ویژه برای binarizer کردن این است که آستانه نمی‌تواند کمتر از صفر باشد.

```
from scipy.sparse import coo
spar = coo.coo_matrix(np.random.binomial(1, .25, 100))
preprocessing.binarize(spar, threshold=-1)
output:ValueError: Cannot binarize a sparse matrix with threshold < 0
```

داده‌های قابل طبقه‌بندی

متغیرهای طبقه‌ای اطلاعات ارزشمندی را ارائه می‌دهند، مانند اینکه در اندیس کتاب‌ها از عدد برای یافتن سریع‌تر اطلاعات استفاده می‌شود. بنابراین ما به وضوح باید متن خود را به عنوان مدل به صورت عدد صحیح نشان دهیم. برای توضیح این بخش از داده‌های iris استفاده می‌کنیم.

```
from sklearn import datasets
import numpy as np
iris = datasets.load_iris()
X = iris.data
y = iris.target
print(y)
output: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2]
```

حال y را به نحوی تغییر می‌دهیم که هر ستون آن معرف یک کلاس باشد (در اینجا فقط ۵ ردیف اول را نمایش می‌دهیم)

```
from sklearn import preprocessing
cat_encoder = preprocessing.OneHotEncoder()
cat_encoder.fit_transform(y.reshape(-1,1)).toarray()[:5]
output: array([[1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.]])
```

در y، ۳ کلاس ۲, ۱, ۰ تعریف شده است حال آرایه‌ای ایجاد کردیم که به تعداد کلاس‌ها (در اینجا سه عدد) ستون دارد. و در هر ستون متناظر با کلاس آن فقط مقدار ۱ قرار می‌دهیم و بقیه ردیف را صفر می‌کنیم. در نهایت یک ماتریس sparse ایجاد می‌گردد.

- اگر گل یک Setosa باشد اولین ستون ۱ و بقیه ستون‌های ردیف صفر است.
- اگر گل چند رنگ باشد ستون دوم ۱ و بقیه ستون‌های ردیف صفر است.

• اگر گل از نوع Virginica باشد ستون سوم ۱ و بقیه ستون‌های ردیف صفر است.

بنابراین، ما می‌توانیم از هر یک از این سه ستون استفاده کنیم تا یک رگرسیون ایجاد نماییم.

کلاس DictVectorizer

گزینه دیگراستفاده از کلاس DictVectorizer است که می‌توان آن را برای تبدیل مستقیم رشته‌ها به ویژگی‌ها استفاده کرد

```
from sklearn.feature_extraction import DictVectorizer
dv = DictVectorizer()
my_dict = [{'species': iris.target_names[i]} for i in y]
dv.fit_transform(my_dict).toarray()[:5]
output: array([[1., 0., 0.],    [1., 0., 0.],    [1., 0., 0.],    [1., 0., 0.],    [1., 0., 0.]])
```

وارد کردن مقادیر از دست رفته (missing values)

درج داده‌های ازدست رفته با مقادیر دیگر imputation نامیده می‌شود که در عمل بسیار مهم است و خوشبختانه روش‌های زیادی برای مقابله با آن وجود دارد. در این دستورالعمل، به چند استراتژی نگاه خواهیم کرد. با این حال، توجه داشته باشید که ممکن است رویکردهای دیگری نیز وجود داشته باشد که با شرایط شما مناسب‌تر باشد. این بدان معنی است که scikit توانایی انجام مفاهیم نسبتاً رایجی را دارد. به سادگی برخی از تغییرات را در داده‌های موجود اعمال و NA (Not Available) را جایگزین می‌کند.

نکته:

محاسبه فرآیند جایگزینی داده‌های از دست رفته با مقادیر جایگزین (imputation) است.

با این حال، اگر داده داده از دست رفته باشد (missing)، و دلیل مشخصی برای این داده وجود دارد (به عنوان مثال، زمان پاسخگویی برای یک سرور که بعد از ۱۰۰ میلی ثانیه تمام می‌شود و همه داده‌ها پردازش نشوند) بهتر است از بسته‌های دیگر مانند PyMC، یا روش‌های آماری که خود می‌توانید پیاده‌سازی کنید استفاده نمایید.

برای توضیحات بیشتر کاری که ما انجام می‌دهیم، ایجاد مقادیر از دست رفته است. در NumPy قابلیتی وجود دارد که این کار را بسیار ساده می‌کند.

```
from sklearn import datasets
import numpy as np
iris = datasets.load_iris()
```

```
iris_X = iris.data
masking_array = np.random.binomial(1, .25,iris_X.shape).astype(bool)
iris_X[masking_array] = np.nan
print(iris_X [:5])
output: array([[5.1, 3.5, 1.4, 0.2],
    [4.9, nan, 1.4, 0.2],
    [nan, 3.2, 1.3, 0.2],
    [4.6, nan, nan, 0.2],
    [5. , nan, 1.4, 0.2]])
from sklearn.impute import SimpleImputer
imputer = SimpleImputer()
iris_X_prime = imputer.fit_transform(iris_X)
iris_X_prime[:5]
output: array([[5.1       , 3.5       , 1.4       , 0.2       ],
    [4.9       , 3.04      , 1.4       , 0.2       ],
    [5.77310924, 3.2       , 1.3       , 0.2       ],
    [4.6       , 3.04      , 3.77672414, 0.2       ],
    [5.        , 3.04      , 1.4       , 0.2       ]])
```

همان طور که مشاهده می‌شود به جای nan مقادیری قرارداده شده است.

این محاسبه با استفاده از استراتژی‌های مختلف کار می‌کند. پیش فرض میانگین است، اما در کل موارد زیر وجود دارد:

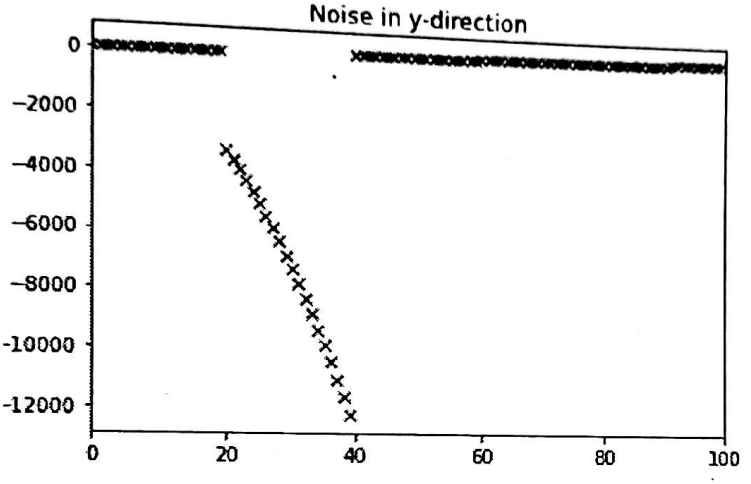
- Mean (میانگین)
- Median (میانه)
- most_frequent (مد)

```
imputer = SimpleImputer(strategy='median')
iris_X_prime = imputer.fit_transform(iris_X)
iris_X_prime[:5]
output:      [4.9, 3. , 1.4, 0.2],      [5.7, 3.2, 1.3, 0.2],      [4.6, 3. , 4.4, 0.2],
[5. , 3. , 1.4, 0.2]])
```

اگر داده‌ها فاقد مقادیر باشند، ممکن است ذاتاً در هر مجموعه داده‌ای (data set) با مقداری خاص نشان داده شده باشد. در مثال قبلی، نحوه انجام این کار، np.nan (مقدار پیش‌فرض از دست رفته) به عنوان مقدار missing مورد استفاده قرار گرفت، اما missing values را می‌توان با مقادیر دیگری نشان داد. شرایطی را در نظر بگیرید که مقادیر از دست رفته 1- باشد. علاوه بر استراتژی محاسبه مقدار از دست رفته، تعیین مقدار از دست رفته برای جایگذار (imputer) نیز امکان‌پذیر است. (پیش‌فرض nan است که مقادیر np.nan را کنترل می‌کند.)

به داده‌ها نویز اضافه کنید و آن‌ها را در متغیری به نام y_noisy ذخیره کنید.

```
y_noisy = y_truth.copy()
#Change y-values of some points in the line
y_noisy[20:40] = y_noisy[20:40] * (-4 * x_vals[20:40]) - 100
plt.title("Noise in y-direction")
plt.xlim([0,100])
plt.scatter(x_vals, y_noisy,marker='x')
```

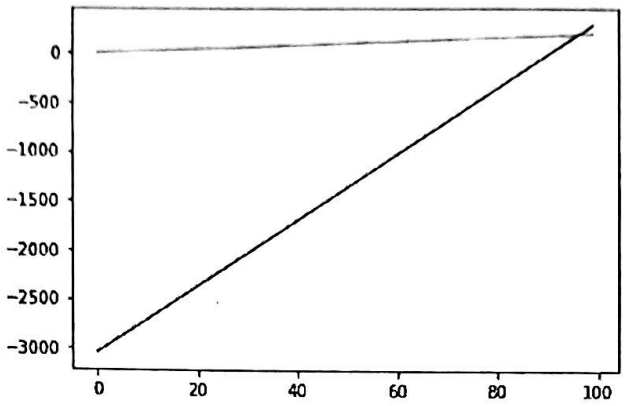


شکل (۲-۵): نویز در داده‌ها

از هر دو LinearRegression و TheilSenRegressor استفاده کنید. با استفاده از خط اصلی به عنوان مجموعه آزمایش، y_truth را به عنوان داده‌های تست یا حقیقی (truth) استفاده نمایید.

```
from sklearn.linear_model import LinearRegression,TheilSenRegressor
from sklearn.metrics import r2_score, mean_absolute_error
named_estimators = [('OLS ', LinearRegression()), ('TSR
',TheilSenRegressor())]
for num_index, est in enumerate(named_estimators):
 y_pred = est[1].fit(x_vals.reshape(-1,1),y_noisy).predict(x_vals.reshape(-1, 1))
print (est[0], "R-squared: ", r2_score(y_truth, y_pred), "Mean Absolute Error",
mean_absolute_error(y_truth, y_pred))
        plt.plot(x_vals, y_pred, label=est[0])
```

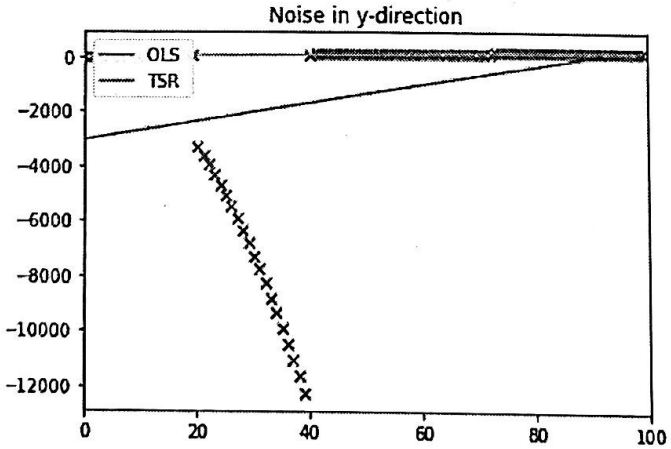
```
OLS  R squared:    904.7565621836717 Mean Absolute Error 1480.9397011701171
TSR  R squared:    0.9999999999265342 Mean Absolute Error 0.0004425033845748491
```



شکل (۲-۶)

حال داده‌ها را تخمین می‌زنیم

```
for num_index, est in enumerate(named_estimators):
    y_pred = est[1].fit(x_vals.reshape(-1,1),y_noisy).predict(x_vals.reshape(-1, 1))
    plt.plot(x_vals, y_pred, label=est[0])
    plt.legend(loc='upper left')
    plt.title("Noise in y-direction")
    plt.xlim([0,100])
    plt.scatter(x_vals, y_noisy,marker='x', color='red')
```



شکل (۲-۷): نمودار با داده‌های نویز

TheilSenRegressor یک تخمین زننده قوی است که در حضور داده‌های پرت عملکرد خوبی دارد. این روش از میانه استفاده می‌کند، که نسبت به داده‌های پرت حساس نیست. در رگرسیون Ordinary least squares (OLS)، مربع خطاها در نظر گرفته می‌شود؛ و می‌تواند نتایج خوب را کاهش دهد.

در ادامه چند نمونه تخمین زننده دیگر آورده شده است

```
from sklearn.linear_model import Ridge, LinearRegression,TheilSenRegressor,
RANSACRegressor, ElasticNet, HuberRegressor
from sklearn.metrics import r2_score, mean_absolute_error
named_estimators = [('OLS ', LinearRegression()),('Ridge ', Ridge()),('TSR ',
TheilSenRegressor()),('RANSAC',
RANSACRegressor()),('ENet ',ElasticNet()),('Huber ',HuberRegressor())]
for num_index, est in enumerate(named_estimators):
        y_pred = est[1].fit(x_vals.reshape(-
1,1),y_noisy).predict(x_vals.reshape(-1, 1))
        print (est[0], "R-squared: ", r2_score(y_truth, y_pred), "Mean Absolute
Error", mean_absolute_error(y_truth, y_pred))
```

```
output: OLS  R-squared:  -904.7565621836717 Mean Absolute Error
1480.9397011701171
  Ridge  R-squared:  -904.7501604538695 Mean Absolute Error
1480.9385250222022
  TSR  R-squared:  0.9999999999265342 Mean Absolute Error
0.000442503384574849
  RANSAC R-squared:  1.0 Mean Absolute Error 1.0236256287043944e-14
  ENet  R-squared:  -904.4272711687513 Mean Absolute Error
1480.8791832083955
  Huber  R-squared:  0.9999999867470813 Mean Absolute Error
0.006644956553308796
```

همانطور که مشاهده می‌شود، Theil-Sen، (RANSAC) وHuber در مقایسه با سایر رگرسیون‌ها عملکرد بهتری در حضور داده‌های پرت دارند.

قرار دادن همه در pipeline

اکنون که با خطوط لوله و تکنیک‌های تبدیل داده آشنا شدید، نمونه پیچیده‌تری را که چندین دستورالعمل قبلی را در خط لوله ترکیب می‌کند، مرور خواهیم کرد. در مبحث قبل، از آن برای مقادیر از دست رفته استفاده می‌کردیم، که فقط یک راه سریع بود. اکنون،

چندین مرحله پیش پردازش را با هم زنجیره خواهیم کرد تا نشان دهیم چگونه خطوط لوله می‌توانند کارهای اضافی را حذف کنند.

```
from sklearn.datasets import load_iris
from sklearn.datasets import load_iris
import numpy as np
iris = load_iris()
iris_data = iris.data
mask = np.random.binomial(1, .25, iris_data.shape).astype(bool)
iris_data[mask] = np.nan
iris_data[:5]
output: array([[5.1, 3.5, nan, 0.2],
       [4.9, nan, nan, 0.2],
       [4.7, 3.2, nan, nan],
       [4.6, 3.1, nan, 0.2],
       [5. , 3.6, 1.4, 0.2]])
from sklearn import pipeline, preprocessing, decomposition
from sklearn.impute import SimpleImputer
pca = decomposition.PCA()
imputer = SimpleImputer()
pipe = pipeline.Pipeline([('imputer', imputer), ('pca', pca)])
iris_data_transformed = pipe.fit_transform(iris_data)
iris_data_transformed[:5]
output:
array([[-0.58147261, -0.90862681,  0.70660157, -0.40447845],
       [-0.62243465, -1.16314017,  0.4820729 , -0.07379522],
       [-0.36850405, -0.87752073, -0.42591685, -0.4897046 ],
       [-0.71888081, -1.39382592,  0.3694514 , -0.19946487],
       [-2.81758508,  0.03038568,  0.29578233, -0.30770416]])
```

در مثال قبل خطوط لوله در سطح بسیار بالایی استفاده شد، اما در واقعیت نیاز است که تغییراتی در پارامترها اعمال گردد. با استفاده از متد set_paramsmethod می‌توان به خصوصیات هر شی در pipelinecan دسترسی پیدا کرد.

حال می‌خواهیم که تعداد اجزا اصلی pca را ۲ در نظر بگیریم و دستور آن به صورت زیر است

```
pipe2.set_params(pca__n_components=2)
pipeline.Pipeline(steps=[('imputer', SimpleImputer(
copy=True,missing_values='NaN', strategy='mean', verbose=0)), ('pca',
PCA(copy=True,
```

```
iterated_power='auto', n_components=2, random_state=None,svd_solver='auto',
tol=0.0, whiten=False))])
iris_data_transformed3 = pipe2.fit_transform(iris_data)
iris_data_transformed3[:5]
output:
array([[-0.58147261, -0.90862681],      [-0.62243465, -1.16314017],      [-
0.36850405, -0.87752073],
       [-0.71888081, -1.39382592],      [-2.81758508,  0.03038568]])
```

گاوسی برای رگرسیون

در این بخش، ما از فرآیند Gaussian برای رگرسیون استفاده خواهیم کرد. در مدل خطی، خواهیم دید که چگونه نمایش اطلاعات قبلی در مورد ضرایب با استفاده از رگرسیون خط بیزی امکان‌پذیر است. در گاوسی، آنچه اهمیت دارد واریانس است و نه میانگین. در گاوسی میانگین صفر است، بنابراین مشخص نمودن تابع کوواریانس اهمیت دارد. در تنظیمات اولیه باید یک پیشین (prior) را روی ضرایب در یک مسئله رگرسیون معمولی اعمال نمود. با یک فرایند گاوسی، می‌توان مقدماتی را در فرم عملکردی داده‌ها قرار داد؛ و این کوواریانس بین نقاط داده است که برای مدل‌سازی داده‌ها استفاده می‌شود که باید در داده‌ها برازش شود.

یک مزیت بزرگ گاوسی این است که آن‌ها می‌توانند به طور احتمالی پیش‌بینی کنند: شما می‌توانید مرزهای اطمینان نسبت به پیش‌بینی‌های خود را بدست آورید. علاوه بر این، پیش‌بینی می‌تواند مشاهدات مربوط به هسته‌های موجود را دخیل نماید: پیش‌بینی‌های حاصل از رگرسیون دارای مرزهای صاف هستند.

```
from sklearn.datasets import load_boston
boston = load_boston()
boston_X = boston.data
boston_y = boston.target
train_set = np.random.choice([True, False], len(boston_y),p=[.75, .25])
from sklearn.gaussian_process import GaussianProcessRegressor
gpr = GaussianProcessRegressor()
```

```
GaussianProcessRegressor(alpha=1e-10,
copy_X_train=True,kernel=None,n_restarts_optimizer=0,
normalize_y=False,optimizer='fmin_l_bfgs_b', random_state=None)
```

چند پارامتر مهم وجود دارد که باید تنظیم شود:

- alpha: پارامتر نویز است و می‌توانید مقدار نویز را برای همه مشاهدات یکسان یا با برای هر مشاهده مقدار نویز را مشخص نمود (بوسیله آرایه numpy).
- kernel: هسته‌ای است که یک تابع را تقریب می‌زند.
- normalize_y: اگر میانگین مجموعه هدف صفر نباشد، می‌توانید آن را درست تنظیم کنید. اگر آن را روی false تنظیم کنید، باز هم کاملاً خوب کار می‌کند.
- n_restarts_optimizer: این منغیر بیانگر تعداد تکرار برای بهینه‌سازی هسته است و بهتر است بین ۱۰ تا ۲۰ تنظیم گردد.

توابع هسته مورد نیاز را وارد می‌کنیم و یک هسته انعطاف‌پذیر تنظیم می‌گردد

```
from sklearn.gaussian_process.kernels import RBF, ConstantKernel as CK
mixed_kernel = kernel = CK(1.0, (1e-4, 1e4)) * RBF(10, (1e-4, 1e4))
```

در آخر، الگوریتم را نمونه‌سازی و متناسب کنید. توجه داشته باشید که آلفا برای همه مقادیر روی ۵ تنظیم شده است.

```
gpr = GaussianProcessRegressor(alpha=5,n_restarts_optimizer=20,kernel =
mixed_kernel)
gpr.fit(boston_X[train_set],boston_y[train_set])
```

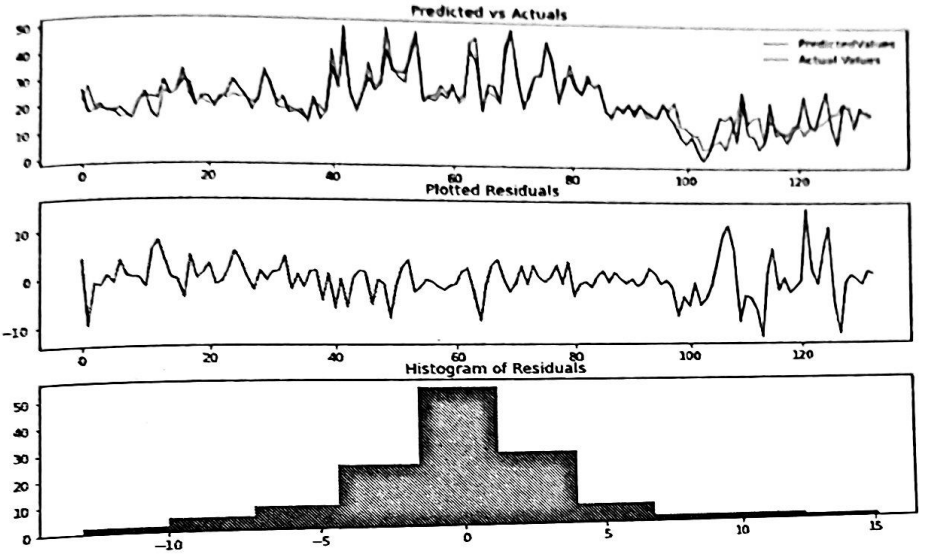
پیش‌بینی‌ها را روی داده‌های دیده نشده به عنوان test_preds ذخیره کنید

```
test_preds = gpr.predict(boston_X[~train_set])
```

حال آن را نمایش می‌دهیم

```
from sklearn.model_selection import cross_val_predict
from matplotlib import pyplot as plt
%matplotlib inline
f, ax = plt.subplots(figsize=(10, 7), nrows=3)
f.tight_layout()
ax[0].plot(range(len(test_preds)), test_preds,label='PredictedValues');
ax[0].plot(range(len(test_preds)),
boston_y[~train_set],label='Actual Values');
ax[0].set_title("Predicted vs Actuals")
ax[0].legend(loc='best')
```

```
ax[1].plot(range(len(test_preds)),test_preds -boston_y[~train_set]);
ax[1].set_title("Plotted Residuals")
ax[2].hist(test_preds - boston_y[~train_set]);
ax[2].set_title("Histogram of Residuals")
```



شکل (۲-۸): پیش‌بینی‌های انجام شده

اعمال cross-validation با پارامترهای نویز

ممکن است از خود بپرسید که آیا این بهترین پارامتر است، یعنی آلفا = ۵؟ برای پی بردن به این موضوع، چند اعتبارسنجی را امتحان کنید.

ابتدا با آلفا = ۵ شروع می‌کنیم

```
from sklearn.model_selection import cross_val_score
gpr5 = GaussianProcessRegressor(alpha=5,
n_restarts_optimizer=20,
kernel = mixed_kernel)
scores_5 = (cross_val_score(gpr5,
boston_X[train_set],
boston_y[train_set],
cv = 4,
scoring = 'neg_mean_absolute_error'))

def score_mini_report(scores_list):
   print ("List of scores: ", scores_list)
   print ("Mean of scores: ", scores_list.mean())
   print ("Std of scores: ", scores_list.std())
```

```
score_mini_report(scores_5)
output: List of scores:  [ -3.90755049  -4.84877728  -4.08272343 -
13.95636848]
Mean of scores:  -6.698854918995426
Std of scores:  4.205050973151173
```

همان طور که مشاهده می‌کنید، مقدار آخرین fold با سه مقدار دیگر متفاوت است.

حال با alpha=7 همان کار را انجام می‌دهیم

```
#Now produce a report with alpha=7
gpr7 = GaussianProcessRegressor(alpha=7,
n_restarts_optimizer=20,
kernel = mixed_kernel)
scores_7 = (cross_val_score(gpr7,
boston_X[train_set],
boston_y[train_set],
cv = 4,
scoring = 'neg_mean_absolute_error'))
score_mini_report(scores_7)
output: List of scores:  [ -3.32331854  -4.28605455  -4.20789276 -13.38783334]
Mean of scores:  -6.301274796331118
Std of scores:  4.108859220832668
```

این مقدارکمی بهتر به نظر می‌رسد. اکنون، alpha = 7 و normalize_y=True را امتحان کنید

```
from sklearn.model_selection import cross_val_score
gpr7n = GaussianProcessRegressor(alpha=7,
n_restarts_optimizer=20,
kernel = mixed_kernel,
normalize_y=True)
scores_7n = (cross_val_score(gpr7n,
boston_X[train_set],
boston_y[train_set],
cv = 4,
scoring = 'neg_mean_absolute_error'))
score_mini_report(scores_7n)
output: List of scores:  [-5.4758255  -6.94384718 -7.01463543 -9.83271116]
Mean of scores:  -7.3167548170094125
Std of scores:  1.5771329438258204
```

خروجی حاصل بهتر به نظر می‌رسد، زیرا میانگین بالاتر و انحراف معیار کمتر است.

آخرین مدل را برای آموزش نهایی انتخاب می‌کنیم

```
gpr7n.fit(boston_X[train_set],boston_y[train_set])
```

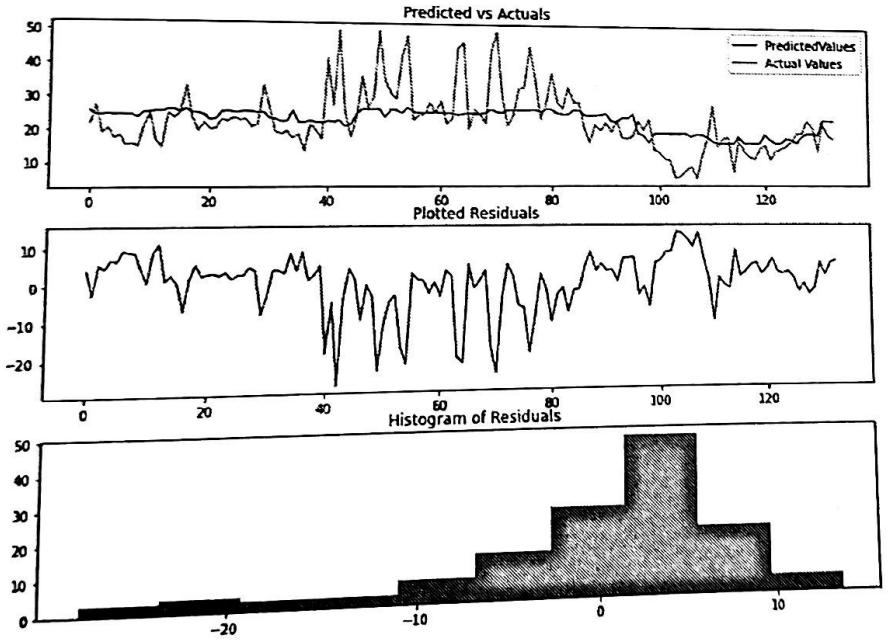
```
output: GaussianProcessRegressor(alpha=7, kernel=1**2 *
RBF(length_scale=10),                    n_restarts_optimizer=20,
normalize_y=True)
```

حال پیش‌بینی را انجام می‌دهیم

```
test_preds = gpr7n.predict(boston_X[~train_set])
```

و آن را نمایش می‌دهیم

```
f, ax = plt.subplots(figsize=(10, 7), nrows=3)
f.tight_layout()
ax[0].plot(range(len(test_preds)), test_preds,label='PredictedValues');
ax[0].plot(range(len(test_preds)),
boston_y[~train_set],label='Actual Values');
ax[0].set_title("Predicted vs Actuals")
ax[0].legend(loc='best')
ax[1].plot(range(len(test_preds)),test_preds -boston_y[~train_set]);
ax[1].set_title("Plotted Residuals")
ax[2].hist(test_preds - boston_y[~train_set]);
ax[2].set_title("Histogram of Residuals")
```



شکل (۲-۹): پیش‌بینی‌های انجام شده

باقیمانده‌ها کمی متمرکز به نظر می‌رسند. همچنین می‌توانید یک آرایه NumPy برای آلفا عبور دهید و ادامه را مانند موارد قبلی انجام دهید.

```
gpr_new =
GaussianProcessRegressor(alpha=boston_y[train_set]/4,n_restarts_optimizer=2
0,kernel = mixed_kernel)
```

استفاده از SGD برای رگرسیون

SGD یک قهرمان ناشناخته در یادگیری ماشین است. در بسیاری از الگوریتم ها، SGD وجود دارد. به دلیل سادگی و سرعتش محبوبیت بالایی دارد- این هر دو چیز خوبی است که می‌توانید هنگام کار با داده‌های زیاد از آن‌ها بهره ببرید. نکته خوب دیگر در مورد SGD این است که این هسته در بسیاری از الگوریتم‌های یادگیری ماشین وجود دارد، زیرا روند را به راحتی توصیف می‌کند.

اگر SGD در مجموعه داده‌های بزرگ عملکرد خوبی دارد، بیایید آن را روی یک مجموعه داده نسبتاً بزرگ آزمایش کنیم.

```
from sklearn.datasets import make_regression
X, y = make_regression(int(1e6)) #1,000,000 rows
```

این خروجی می‌تواند به سیستم وابسته باشد، بنابراین ممکن است نتایج یکسانی نداشته باشید

```
print ("{:,}".format(X.nbytes))
output: 800,000,000
```

برای دستیابی به دیدگاه قابل فهم‌تر، بایت‌ها را به مگابایت تبدیل کنیم. تقریباً ۱ میلیون بایت در یک مگابایت وجود دارد.

```
X.nbytes / 1e6
```

بنابراین، تعداد بایت‌ها در هر نقطه داده به شرح زیر است

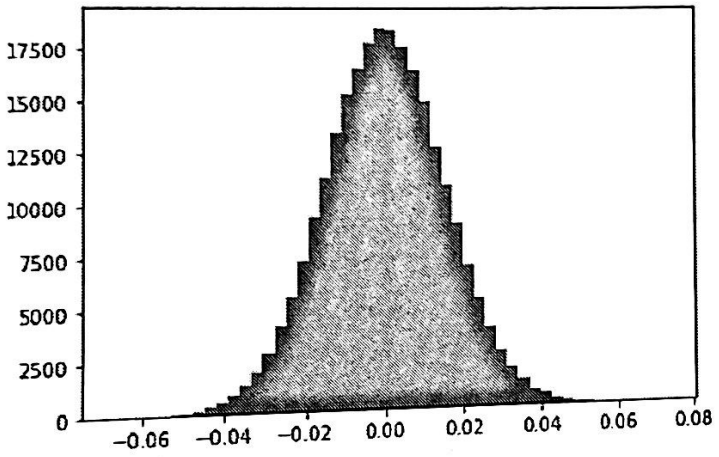
```
X.nbytes / (X.shape[0]*X.shape[1])
Output: 8.0
```

حال SGDRegressor را اعمال می‌کنیم

```
from sklearn.linear_model import SGDRegressor
sgd = SGDRegressor()
train = np.random.choice([True, False], size=len(y), p=[.75, .25])
sgd.fit(X[train], y[train])
SGDRegressor(alpha=0.0001, average=False, epsilon=0.1, eta0=0.01,
fit_intercept=True, l1_ratio=0.15,
learning_rate='invscaling',
loss='squared_loss', penalty='l2', power_t=0.25,
random_state=None, shuffle=True, verbose=0,
warm_start=False)
```

مهم‌ترین چیزی که اکنون باید بدانید این است که تابع خطا ما "مربع خطا" است. همانند آنچه که در رگرسیون خطی اتفاق می‌افتد. همچنین لازم به ذکر است که درهم‌سازی (shuffling) را باید اعمال نماییم، زیرا این عمل برای از بین بردن همبستگی بالقوه مفید است. در آرایه X، scikit به طور خودکار ستونی از یک‌ها را اعمال می‌کند و نمودار مربوطه نرمال می‌باشد. (شکل ۲-۱۰)

```
y_pred = sgd.predict(X[~train])
%matplotlib inline
plt.hist(y[~train] - y_pred , bins=50)
plt.show()
```



شکل (۲-۱۰): توزیع نرمال

واضح است که مجموعه داده مورد استفاده ما خیلی بد نبود، اما شما می‌توانید مجموعه داده‌هایی با اندازه بزرگ‌تر را تصور نمایید. به عنوان مثال، اگر روزی در وال استریت کار می‌کردید، ممکن است ۲ میلیارد تراکنش در هر صرافی بازار انجام شود. حال، تصور کنید که داده‌های یک هفته یا یک سال دارید. الگوریتم‌های هسته‌ای با حجم عظیمی از داده‌ها به خوبی کار نمی‌کنند. دلیل این امر این است که برای انجام SGD، باید در هر مرحله شیب را محاسبه کنیم. اصل الگوریتم این است که در هر مرحله، ما یک سری ضریب جدید را محاسبه و این را با نرخ یادگیری و نتیجه تابع هدف به روز کنیم. این ممکن است مانند شبه کد زیر باشد

```
while not converged:
w = w –learning_rate*gradient(cost(w))
```

متغیرهای مربوط به شرح زیر است:

- w: ماتریس coefficient است.
- learning_rate: این نشان می‌دهد که هر گام در هر تکرار چقدر بزرگ است. اگر به همگرایی خوبی دست پیدا نکنید، ممکن است لازم باشد آن را تنظیم نمایید.
- gradient: این ماتریس مشتقات دوم است.
- cost: این خطای مربع برای رگرسیون است.

این امر چندان بد نخواهد بود، به جز این واقعیت که تابع گرادیان بسیار پرهزینه است. با بزرگ‌تر شدن بردار ضرایب، محاسبه شیب بسیار سخت می‌شود. برای هر مرحله به‌روزرسانی، باید وزن جدیدی را برای هر نقطه از داده‌ها محاسبه کرده و سپس به‌روز کنیم.

SGD کمی متفاوت عمل می‌کند. به جای تعریف قبلی برای شیب نزول دسته ای، ما پارامتر را با هر نقطه داده جدید به‌روز می‌کنیم. این نقطه به صورت تصادفی انتخاب می‌شود، از این‌رو "شیب نزول تصادفی" نامگذاری شده است. نکته آخر در مورد SGD این است که این یک عمل فرا ابتکاری است و به بسیاری از الگوریتم‌های یادگیری ماشین قدرت زیادی می‌بخشد.